

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

Aplicaciones Distribuidas (ISR-701) – Unidad 3: Bases de Datos Distribuidas

Informe Técnico Individual

Análisis de Consistencia y Replicación en Bases de Datos Distribuidas

¿Es el modelo de consistencia elegido el óptimo para el dominio del sistema?

Código:	TA-IND-02
Tipo:	Trabajo Autónomo – Individual – Sumativa
Unidad:	3 – Bases de Datos Distribuidas
Equipo PFC analizado:	ACC – Soporte Técnico ISP (gestión de tickets)
Estudiante:	Jeremy Alexis Alvarez Parraga
Docente:	Prof. Guerrero Ulloa G.C.
Fecha:	14 de julio de 2026

Repositorio: <https://github.com/DeJere/TA-IND-02-Informe-Tcnico-Individual-.git>

Resumen

El equipo ACC del Proyecto Fin de Curso (PFC) implementa un sistema de gestión de tickets de soporte técnico para un proveedor de Internet (ISP) organizado en tres zonas operativas. Este informe caracteriza el modelo de consistencia realmente implementado en dicho sistema: un núcleo transaccional sobre CockroachDB, particionado geográficamente por zona y replicado de forma síncrona mediante consenso Raft bajo aislamiento serializable, combinado con una coreografía de eventos asíncrona sobre Apache Kafka entre **ticket-service** y **ai-service** que introduce consistencia eventual en el enriquecimiento de clasificación automática de tickets. Se ubica esta arquitectura híbrida en el espectro de consistencia de Viotti y Vukolić, se interpreta la decisión de diseño bajo los marcos CAP y PACELC, y se contrasta el modelo actual con dos alternativas – réplica primario–secundario asíncrona y almacenamiento sin líder por quorum – mediante cinco criterios técnicos explicitados en una tabla comparativa. La postura sostenida es que el modelo híbrido resulta adecuado para el dominio porque separa la consistencia fuerte que exigen los invariantes críticos del negocio (asignación de técnicos, cumplimiento de SLA) de la consistencia relajada que es tolerable en una tarea de enriquecimiento no bloqueante, si bien persisten limitaciones de latencia de escritura entre zonas y de observabilidad del retraso de la coreografía.

1. Introducción

El Proyecto Fin de Curso del equipo ACC aborda el dominio de *Soporte Técnico ISP*: la gestión del ciclo de vida de tickets de soporte técnico para un proveedor de Internet que opera en tres zonas geográficas de la ciudad de Quevedo (centro, norte y sur). Cada ticket se origina cuando un cliente reporta una incidencia (conectividad, DNS, hardware, configuración o velocidad), se le asigna una prioridad y un técnico de la zona correspondiente, y su resolución queda sujeta a un acuerdo de nivel de servicio (SLA) cuyo incumplimiento debe poder detectarse de forma confiable. El sistema está compuesto por tres microservicios (**auth-service**, **ticket-service** y **ai-service**), un clúster distribuido de tres nodos de CockroachDB y un backbone de mensajería Apache Kafka que conecta la creación de tickets con un servicio de clasificación automática basado en IA.

Un sistema de esta naturaleza no puede tratarse como una única base de datos monolítica: la distribución geográfica de los técnicos y clientes, la necesidad de tolerar la caída de un nodo sin detener el servicio, y la existencia de una tarea de enriquecimiento (la clasificación automática) que no debería bloquear el flujo principal de negocio, obligan a tomar decisiones explícitas sobre qué garantías de consistencia ofrecer y cómo replicar los datos. Estas decisiones no son gratuitas: cada incremento en la fuerza de la consistencia impone mayor coordinación entre réplicas y, por lo tanto, mayor latencia y menor disponibilidad ante fallos o particiones de red [6].

El objetivo de este informe es doble. Primero, caracterizar con precisión el modelo de consistencia y la estrategia de replicación efectivamente implementados en el PFC del equipo ACC, a partir de la configuración real del clúster de CockroachDB y de la arquitectura de mensajería entre servicios. Segundo, evaluar de forma crítica – a la luz de los teoremas CAP y PACELC y de al menos dos alternativas de diseño – si dicho modelo constituye la elección óptima para el dominio de soporte técnico de un ISP, o si existen compromisos más adecuados que el equipo debería considerar.

2. Marco teórico

2.1. Modelos de consistencia

La consistencia de réplicas no es una propiedad binaria sino un espectro de garantías con semántica formal precisa, ordenadas por su fuerza [6]. En el extremo más fuerte, la **linealizabilidad** (o consistencia fuerte) exige que toda operación parezca ejecutarse de forma atómica en un instante único dentro del tiempo real transcurrido entre su invocación y su respuesta, de modo que cualquier lector observe siempre el efecto de la última escritura confirmada. La **consistencia secuencial** relaja la referencia al tiempo real pero preserva un orden total único de operaciones, visible igual para todos los procesos. La **consistencia causal** exige preservar únicamente el orden entre operaciones causalmente relacionadas, permitiendo reordenar operaciones concurrentes e independientes. La **consistencia de lectura de los propios cambios** (read-your-writes) es una garantía todavía más débil, centrada en la sesión: un proceso siempre ve sus propias escrituras, aunque no necesariamente las de otros procesos de forma inmediata. Finalmente, la **consistencia eventual** solo garantiza que, en ausencia de nuevas escrituras, todas las réplicas convergerán eventualmente al mismo estado, sin acotar cuánto tiempo tarda esa convergencia.

Esta jerarquía tiene un correlato operacional directo: un modelo más fuerte requiere que las réplicas coordinen cada operación (típicamente mediante consenso distribuido) antes de confirmarla, lo cual incrementa la latencia y reduce la disponibilidad cuando alguna réplica no es alcanzable; un modelo más débil permite confirmar operaciones localmente y propagar los cambios de forma diferida, ganando disponibilidad y latencia a costa de exponer estados transitoriamente divergentes entre réplicas.

2.2. Teoremas de compromiso: CAP y PACELC

El teorema CAP, formalizado por Gilbert y Lynch a partir de la conjetura de Brewer, establece que ante una partición de red un sistema distribuido no puede garantizar simultáneamente consistencia (C) y disponibilidad (A); debe sacrificar una de las dos mientras persista la partición, conservando siempre la tolerancia a particiones (P) como requisito irrenunciable en cualquier sistema realmente distribuido [4]. Sin embargo, CAP describe únicamente el comportamiento del sistema durante una partición, un evento relativamente infrecuente; no dice nada sobre el compromiso que enfrenta el sistema en operación normal, que es el caso dominante en la práctica.

Ese vacío lo cubre la formulación PACELC de Abadi: si hay Partición (P), el sistema elige entre Disponibilidad (A) y Consistencia (C), igual que en CAP; pero en su defecto (E, *else*), en operación normal sin particiones, el sistema elige entre Latencia (L) y Consistencia (C) [1]. PACELC captura así que incluso sistemas que jamás sufren una partición real deben decidir permanentemente cuánta latencia están dispuestos a pagar por mantener las réplicas sincronizadas. Un sistema PC/EC (como Spanner) prioriza consistencia tanto ante particiones como en operación normal, aceptando mayor latencia; un sistema PA/EL (como Dynamo) prioriza disponibilidad y baja latencia en ambos casos. La decisión de diseño de cualquier PFC debe leerse bajo ambos ejes, no solo bajo el eje de partición de CAP.

2.3. Estrategias de replicación

La estrategia de replicación determina cómo se propagan las escrituras entre réplicas y, con ello, condiciona directamente el modelo de consistencia alcanzable [5]. En la **replicación primario-respaldo** (*leader-follower*) todas las escrituras pasan por un nodo líder que las propaga a los seguidores; si la propagación es síncrona (el líder espera confirmación de un quórum de seguidores antes de responder al cliente) se obtienen garantías fuertes a costa de latencia, mientras que si es asíncrona se gana disponibilidad y latencia a costa de una ventana de inconsistencia y de posible pérdida de datos ante la caída del líder. La **replicación multi-líder** admite varios nodos que aceptan escrituras de forma concurrente, lo que exige resolución de conflictos y habitualmente relaja la consistencia. La **replicación sin líder por quorum** elimina la figura de líder fijo: cualquier réplica puede aceptar escrituras, y la consistencia percibida depende de los parámetros de quórum de lectura y escritura (R y W) frente al total de réplicas N .

Los sistemas de referencia industrial ilustran los extremos de este espectro. Dynamo, de Amazon, prioriza la disponibilidad mediante replicación sin líder con quórum ajustable y resolución de conflictos por vectores de versión, aceptando consistencia eventual como consecuencia directa de esa elección [3]. Spanner, de Google, se ubica en el extremo opuesto: ofrece consistencia externa (transacciones globalmente consistentes y linealizables) apoyándose en consenso Paxos por fragmento de datos y en sincronización temporal de alta precisión (TrueTime) para ordenar transacciones distribuidas sin coordinación adicional [2]. CockroachDB, la base de datos empleada en el PFC del equipo ACC, se inspira explícitamente en el diseño de Spanner mientras que Kafka, empleado como backbone de mensajería asíncrona, se aproxima en espíritu al extremo de disponibilidad que ilustra Dynamo.

3. Análisis del modelo de consistencia del PFC

3.1. Caracterización del modelo implementado

El sistema del equipo ACC no implementa un único modelo de consistencia uniforme, sino dos regiones con garantías distintas, separadas por la frontera asíncrona de Kafka. La Figura 1 representa la topología real desplegada.

Núcleo transaccional (tickets, técnicos, autenticación). El clúster de CockroachDB se despliega con tres nodos (`roach1`, `roach2`, `roach3`), cada uno anclado a una *locality* distinta (`zone=quevedo-centro`, `zone=quevedo-norte`, `zone=quevedo-sur`) mediante el flag `-locality` en el arranque de cada nodo. La tabla principal `tickets` se particiona horizontalmente por lista (`PARTITION BY LIST`) sobre la columna `zone`, con una partición por zona operativa (`tickets_centro`, `tickets_norte`, `tickets_sur`) y una partición por defecto. Cada partición se configura explícitamente con `num_replicas = 3` y una restricción de localidad preferente hacia su zona (`constraints = {"+zone=quevedo-centro": 1}`, análoga para las otras dos), de modo que el *leaseholder* de cada rango tiende a ubicarse en el nodo de su propia zona, pero las tres réplicas Raft de cada rango se mantienen sincronizadas entre los tres nodos del clúster. Este es, por definición, un esquema de **replicación líder-seguidor síncrona por rango**: cada escritura debe alcanzar un quórum Raft (dos de tres réplicas) antes de confirmarse, y CockroachDB expone únicamente aislamiento `SERIALIZABLE`, la garantía más fuerte del espectro descrito en

la Sección 2.1 para operaciones transaccionales sobre el mismo rango. Tanto `auth-service` como `ticket-service` se conectan a este clúster con `ddl-auto: none`, delegando el esquema – incluida la partición – a los scripts de `db-cluster/scripts`, y sin contraseña (modo `-insecure`, aceptable para el entorno de desarrollo del PFC pero no para un despliegue productivo).

Coreografía de clasificación (borde asíncrono). Cuando `ticket-service` crea un ticket, publica un evento en el tópico `ticket.created` de Kafka. `ai-service` (un servicio Python/FastAPI) consume ese tópico, ejecuta la clasificación y publica el resultado en `ticket.classified`; `ticket-service` consume ese segundo tópico y actualiza el ticket con la categoría inferida. Este patrón – una *saga por coreografía* sin coordinador central – desacopla completamente ambos servicios: no existe transacción distribuida ni bloqueo entre la creación del ticket y su clasificación. La consecuencia directa es que, entre el instante en que el ticket se crea (y ya es visible, sin categoría, para cualquier lector del núcleo transaccional) y el instante en que `ticket-service` aplica la actualización proveniente de `ticket.classified`, existe una ventana de **inconsistencia eventual** sobre el campo `category`: dos lectores concurrentes del mismo ticket pueden observar temporalmente valores distintos de ese campo según si su lectura ocurre antes o después de que la coreografía complete su ciclo. Kafka, en la configuración de un solo bróker usada en el PFC, no aporta por sí mismo garantías de consistencia fuerte entre productor y consumidores; su contrato es de entrega ordenada por partición con procesamiento asíncrono, que es precisamente la propiedad que hace posible desacoplar la disponibilidad del flujo principal de tickets respecto de la disponibilidad del servicio de IA.

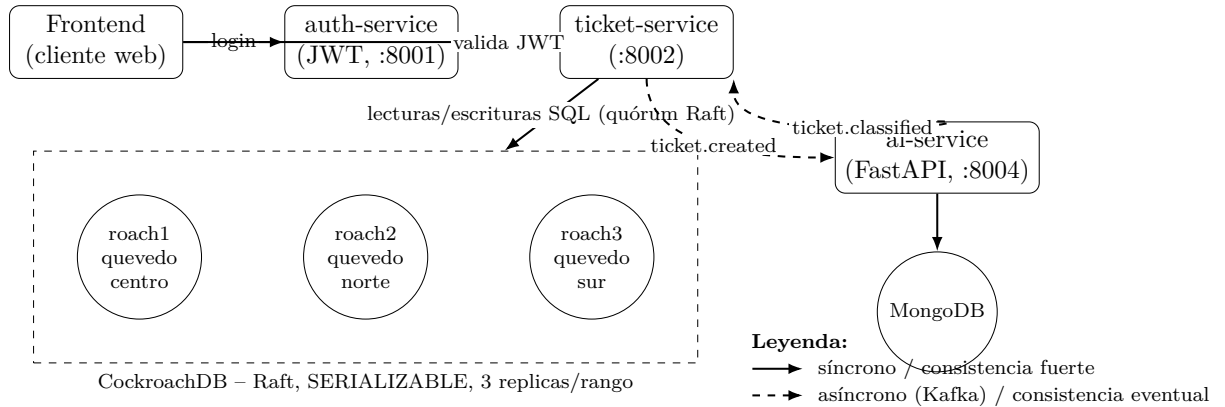


Figura 1: Topología de replicación y consistencia del PFC del equipo ACC. El núcleo transaccional (izquierda, gris punteado) usa replicación síncrona por consenso Raft con aislamiento serializable; la coreografía con `ai-service` (derecha) atraviesa la frontera asíncrona de Kafka y opera bajo consistencia eventual.

3.2. Justificación técnica ligada al dominio

La coexistencia de dos regiones de consistencia distintas no es una inconsistencia de diseño, sino una lectura implícita – y correcta – de PACELC aplicada a dos subproblemas del mismo dominio con requisitos opuestos.

Para el núcleo transaccional, el dominio impone invariantes que toleran mal la consistencia relajada. La asignación de un técnico a un ticket, el cómputo del cumplimiento de un SLA (`sla_deadline`, `sla_breached`) y el estado del ticket (`status`) son datos críticos de negocio: una lectura obsoleta podría llevar a asignar dos técnicos distintos al mismo ticket, o a subestimar

un incumplimiento de SLA con consecuencias contractuales frente al cliente. Bajo PACELC, el equipo ACC eligió **PC/EC**: tanto ante una partición (P) como en operación normal (E), el sistema prioriza consistencia (C) sobre disponibilidad/latencia, aceptando el costo de esperar el quórum Raft en cada escritura. Esta elección es coherente con el hecho de que CockroachDB solo ofrece aislamiento serializable: el equipo no diseñó una relajación deliberada de consistencia para el núcleo, sino que adoptó la garantía más fuerte disponible en la plataforma elegida.

Para la coreografía de clasificación, en cambio, el dominio es tolerante a *staleness*: la categoría automática de un ticket es un dato de enriquecimiento, útil para priorizar la cola de trabajo de los técnicos, pero su ausencia o retraso momentáneo no compromete la corrección del flujo de soporte – el ticket sigue siendo válido, visible y asignable sin categoría. Aquí el equipo eligió implícitamente **PA/EL** para ese subproblema: prioriza disponibilidad y baja latencia del flujo principal (`ticket-service` nunca espera a `ai-service` para responder al cliente) sobre la consistencia inmediata del campo `category`, exactamente el mismo compromiso que justifica el uso de consistencia eventual en Dynamo [3]. La partición geográfica por zona (`PARTITION BY LIST` + restricciones de localidad) responde además a un requisito de negocio real y verificable en el código: la mayoría de las consultas y actualizaciones de un ticket las realiza el técnico de su propia zona, por lo que anclar el *leaseholder* de cada partición al nodo local reduce la latencia de las operaciones zonales más frecuentes sin renunciar a las tres réplicas síncronas que garantizan tolerancia a la caída de cualquier nodo individual.

4. Evaluación de alternativas

Para contrastar el modelo actual se consideran dos alternativas de consistencia/replicación razonables para el mismo dominio de gestión de tickets.

Alternativa 1 – Réplica primario–secundario asíncrona (estilo MongoDB replica set). Consistiría en reemplazar el clúster particionado de CockroachDB por una única base de datos primaria que replica de forma asíncrona hacia una o más secundarias, permitiendo lecturas desde secundarias para descargar al primario. Es, de hecho, el mismo patrón que el propio PFC ya usa – pero para `ai-service`, no para el núcleo – con la instancia `mongo` del *stack* de mensajería, lo que la vuelve una alternativa concretamente evaluable en este contexto.

Alternativa 2 – Almacenamiento sin líder por quórum (estilo Dynamo/Cassandra). Consistiría en distribuir la tabla `tickets` sobre un anillo de nodos sin líder fijo, con factor de replicación $N = 3$ y parámetros de quórum de lectura/escritura ajustables ($R + W > N$ para consistencia fuerte ajustada, o $R + W \leq N$ para favorecer disponibilidad y latencia), resolviendo conflictos de escrituras concurrentes mediante vectores de versión o *last-write-wins*.

La Tabla 1 contrasta el modelo actual frente a ambas alternativas según cinco criterios técnicos.

Frente a la Alternativa 1, el modelo actual ofrece una garantía estrictamente más fuerte para el núcleo (serializable multi-nodo frente a fuerte-solo-en-el-primario), al costo de una latencia de escritura algo mayor; dado que el dominio penaliza más una asignación incorrecta de técnico que unos pocos milisegundos adicionales por transacción, el modelo actual domina para el núcleo. Frente a la Alternativa 2, el modelo actual sacrifica algo de disponibilidad y de tolerancia a particiones extremas a cambio de eliminar por completo la complejidad de resolución de conflictos concurrentes, complejidad que un equipo de PFC de alcance académico difícilmente podría operar

Tabla 1: Comparación del modelo actual del PFC (ACC) frente a dos alternativas de consistencia/replicación.

Criterio	Modelo actual (CockroachDB Raft + Kafka)	Alt. 1 (primario-secundario asíncrono)	Alt. 2 (sin líder por quórum)
Consistencia	Serializable (fuerte) en el núcleo; eventual solo en el campo de clasificación	Fuerte en el primario; eventual en lecturas de secundarias	Ajustable vía R, W ; eventual por defecto, fuerte solo con $R + W > N$
Disponibilidad ante caída de nodo	Alta: tolera la caída de 1 de 3 nodos sin pérdida de quórum Ra.	Baja para escritura si cae el primario (requiere failover); alta para lectura	Muy alta: cualquier nodo disponible puede servir la operación
Latencia de escritura	Media: espera quórum Raft (2 de 3), atenuada por afinidad de zona	Baja (el primario confirma sin esperar secundarias)	Baja a media, según W configurado
Tolerancia a particiones	Alta dentro del clúster; el núcleo prioriza C sobre A ante partición	El primario aislado sigue escribiendo; secundarias divergen	Alta: diseñado explícitamente para operar particionado (P)
Complejidad operativa	Media: partición y localidades declarativas, sin resolución manual de conflictos	Baja: patrón estándar y bien soportado	Alta: exige gestión explícita de conflictos y de quórum

de forma confiable dentro del tiempo del curso.

5. Discusión crítica

¿Es el modelo de consistencia elegido el óptimo para el dominio del sistema? La respuesta razonada es: **sí, para el núcleo transaccional, con reservas operativas, y sí, de forma casi óptima, para la coreografía de clasificación.**

Para el núcleo, la elección de consistencia serializable vía Raft es defendible bajo PACELC porque el dominio – asignación de técnicos y cómputo de SLA – es exactamente el tipo de carga para la que CAP/PACELC recomienda priorizar C sobre A/L: los costos de una violación de consistencia (doble asignación, SLA mal calculado) son altos y difícilmente reversibles, mientras que el costo de la latencia adicional del quórum Raft es acotado y predecible dentro de un clúster de tres nodos en la misma región. Sin embargo, subsisten dos limitaciones concretas. Primera, el clúster corre en modo `-insecure` y sin réplicas fuera de la máquina de desarrollo: la tolerancia a fallos que ofrece Raft es real frente a la caída de un proceso, pero no frente a un fallo del host completo, lo que en producción exigiría nodos en máquinas o zonas de disponibilidad físicamente independientes. Segunda, anclar el *leaseholder* de cada partición a su zona reduce la latencia de las operaciones locales, pero cualquier consulta que cruce zonas (por ejemplo, un reporte agregado de SLA a nivel de ciudad) sigue pagando el costo de un quórum Raft con al menos una réplica remota, un compromiso que el equipo no documenta explícitamente y que conviene medir.

Para la coreografía de clasificación, la consistencia eventual vía Kafka es la elección correcta en espíritu – el enriquecimiento no debe bloquear el flujo crítico – pero el PFC no implementa

ninguna de las mitigaciones habituales para acotar el impacto de esa eventualidad: no hay un límite explícito de tiempo máximo de convergencia (*staleness bound*), ni una señal visible en la interfaz de que la categoría de un ticket está "pendiente de clasificar", ni una política de reintentos o *dead-letter queue* documentada si `ai-service` cae mientras hay mensajes en `ticket.created`. Estas mitigaciones no cambiarían el modelo de consistencia elegido, pero sí reducirían el riesgo de que la consistencia eventual se perciba, erróneamente, como un dato perdido en vez de un dato en tránsito. En conjunto, el modelo híbrido implementado es una lectura correcta de CAP/PACELC aplicada de forma diferenciada a dos subproblemas con requisitos opuestos dentro del mismo dominio, y su principal margen de mejora no está en cambiar el modelo de consistencia, sino en hacer explícitas y observables las garantías – y las ausencias de garantía – que ya implementa.

6. Conclusiones

El PFC del equipo ACC implementa, de facto, un modelo de consistencia híbrido: consistencia fuerte (serializable, vía consenso Raft con replicación síncrona de tres réplicas por partición geográfica) para el núcleo transaccional de tickets, y consistencia eventual (vía una coreografía asíncrona sobre Kafka) para el enriquecimiento de clasificación automática. Esta combinación no fue necesariamente el resultado de un análisis explícito de CAP/PACELC por parte del equipo, pero es, retrospectivamente, coherente con lo que ambos marcos recomiendan para un dominio donde coexisten invariantes críticos de negocio (asignación, SLA) con tareas de enriquecimiento tolerantes a retraso (clasificación). La comparación con dos alternativas de diseño – réplica primario-secundario asíncrona y almacenamiento sin líder por quórum – confirma que ninguna domina estrictamente al modelo actual para el núcleo transaccional, y que la complejidad adicional de un esquema sin líder no se justifica en el alcance de un PFC académico. El aprendizaje principal para el equipo es que la optimalidad del modelo actual es condicional: se sostiene mientras el dominio siga privilegiando corrección sobre latencia en el núcleo y tolere retraso en el enriquecimiento, y se debilita si el sistema creciera hacia múltiples regiones geográficas reales o si el negocio empezara a exigir garantías de tiempo de convergencia sobre la clasificación automática.

7. Declaración de uso de IA generativa

Para la elaboración de este informe se utilizó Claude (Anthropic), un asistente de IA generativa, como herramienta de apoyo en las siguientes tareas: (i) inspección del código fuente y de los archivos de configuración del PFC del equipo ACC (`docker-compose`, `application.yml`, scripts SQL de partición y zonas) para caracterizar con precisión el modelo de consistencia y la estrategia de replicación realmente implementados; (ii) organización del documento conforme a la estructura obligatoria de la guía TA-IND-02; y (iii) redacción inicial del marco teórico, del análisis del modelo del PFC, de la tabla comparativa y de la discusión crítica. El estudiante revisó el contenido generado, verificó su correspondencia con el código real del PFC y asumió la postura crítica final presentada en la Sección 5.

Referencias

- [1] D. J. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,” *Computer*, vol. 45, no. 2, pp. 37–42, 2012. doi: 10.1109/MC.2012.33
- [2] J. C. Corbett *et al.*, “Spanner: Google’s Globally Distributed Database,” *ACM Transactions on Computer Systems*, vol. 31, no. 3, pp. 1–22, 2013, art. no. 8. doi: 10.1145/2491245
- [3] G. DeCandia *et al.*, “Dynamo: Amazon’s Highly Available Key-value Store,” in *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP ’07)*, 2007, pp. 205–220. doi: 10.1145/1294261.1294281
- [4] S. Gilbert and N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002. doi: 10.1145/564585.564601
- [5] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 3rd ed. distributed-systems.net, 2017. isbn: 978-1543057386.
- [6] P. Viotti and M. Vukolić, “Consistency in Non-Transactional Distributed Storage Systems,” *ACM Computing Surveys*, vol. 49, no. 1, pp. 1–34, 2016, art. no. 19. doi: 10.1145/2926965